

w7L39 Eligibility Traces

In RL first tool of choice: Q-learning and SARSA ->

Aim: reduce sample complexity

Eligibility Traces

$TD(0) \rightarrow TD(\lambda)$

$$G_t^{MC} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T$$
$$G_t^{TD} = R_{t+1} + \gamma U(S_{t+1}) \equiv G_t^{(1)}$$

One step truncated corrected reward = one step return

$$R_{t+1} + \gamma R_{t+2} + \gamma^2 U(S_{t+2}) \equiv G_t^{(2)}$$

Two step return

On-line - $G_t(n)$ wait for n steps

Off-line - wait till end of the episode

Convergence might not be faster in online case

In online - variance will be higher

- Update along the way
- Will use new value function for plugging it in

$$G_t^{TD} = R_{t+1} + \gamma V(s_{t+1}) \equiv G_t^{(1)}$$

$$R_{t+1} + \gamma R_{t+2} + \gamma^2 V(s_{t+2}) \equiv G_t^{(2)}$$

New s_{t+2} is value updated after visiting it

- Works better when you are doing control
- State is updated many time

In offline - will update average update

- You are visiting the state multiple times
- Will use same value first or second time i visit the state
- If trajectory is long - have to wait long
- Only Update for each state

Batch of update (batch mode TD) - end of 6 or 8 step return give best error

- Take all trajectories for episodes
- Better in terms of variance reduction

It is NOT necessary to know the right ans

- But what it depends on

How to debug production code - charlie chaplin - clean track

Problem is 19-state problems (9 on left, 9 on right, start from mid)

- Just do random walk
- Why 5 or 6 should be the point where you reach - what is best n
- GT-100 was bad

Finding best-n is not easy

Every time i make update, will change by

$$\Delta V_t(s)$$

$$V_{t+1}(s) = V_t(s) + \Delta V_t(s)$$

$$\Delta V_t(s) = \alpha [r_{t-n}^{(n)} - V_t(s)]$$

$$V_{t+1}(s) = V_t(s) + \Delta V_t(s)$$

After n time steps for state s - update

Will update state everytime - if not computed update with 0

On-line

off-line

$$\Delta V_t(s) = \alpha [r_{t-n}^{(n)} - V_t(s)]$$

$$V_{t+1}(s) = V_t(s) + \Delta V_t(s)$$

$$V(s) = V(s) + \sum_{t=0}^T \Delta V_t(s)$$

Offline case

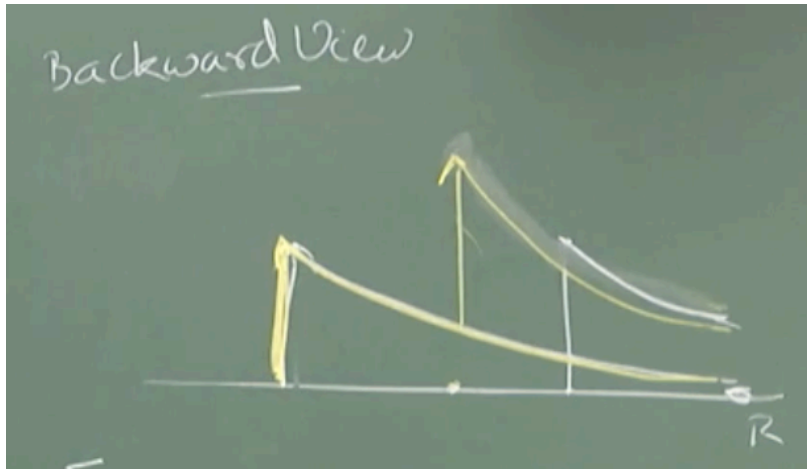
Class of update rules called - Stochastic averaging

- Small deltas - is averaging

What is the right value of n?

Eligibility traces - what stimuli is causing change in behaviour

- This stimuli is eligible for change on behaviour
- Light comes just before food (more proximal to outcome -> more eligible)
- Than bell which range 5 min ago



Backward view of TD lambda - look at when reward happened

Up arrow stimulus

Start trace as time decays - which every stimulus has high

- Spatially closer with - taken by discounted factor
- Temporal proximity - Eligibility mechanism

Yellow occurred twice - if it occurs 3rd time, it will be higher so take it

- Not just do immediate update
- Will update many states which occurred previously also, they also have some responsibility
- Is eligibility mechanism
- What was the notion of value function or return update - explanation

They call it

Forward View of TD(lambda)

- I am in state of $s \rightarrow$ what will be future expected return

Convert n-step return to lambda-return

- It looked arbitrary
- In book explained Forward view

24min: Forward view - new notion of return lambda-return

- Tried to find weighted average of

$$G_t^\lambda = 0.5 G_t^{(1)} + 0.5 G_t^{(2)}$$

Will use some kind of average of two

Gives advantage of bootstrapping and real sample

$$G_t^\lambda = 0.33 G_t^{(1)} + 0.33 G_t^{(2)} + 0.34 G_t^{(3)}$$

Can generalise for ever

$$G_t^\lambda = G_t^{(1)} + \lambda G_t^{(2)} + \lambda^2 G_t^{(3)} + \dots$$

Coefficients should sum to 1

- If not i am over estimating

What to do to make it sum to 1

$$G_t^\lambda = (1-\lambda) \left[G_t^{(1)} + \lambda G_t^{(2)} + \lambda^2 G_t^{(3)} + \dots \right]$$

- Multiply by 1-lambda

$$G_t^\lambda = (1-\lambda) \left[G_t^{(1)} + \lambda G_t^{(2)} + \lambda^2 G_t^{(3)} + \dots \right]$$

$$= (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

Summation will go on for ever, to terminate

$$G_t^\lambda = (1-\lambda) \left[G_t^{(1)} + \lambda G_t^{(2)} + \lambda^2 G_t^{(3)} + \dots \right]$$

$$= (1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_t^{(n)}$$

Add T-t to summation

Still there is a problem

If episode terminates at some point, what about rest of the thing?

- My sum will not be

If $n > T-t-1$

- What is the expected $\rightarrow$ return will not change
- It will be same as MC return G_t

$$G_t^{MC} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T = G_t$$

G_t without superscript is org return

$$G_t^\lambda = (1-\lambda) \left[G_t^{(1)} + \lambda G_t^{(2)} + \lambda^2 G_t^{(3)} + \dots \right]$$
$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_t^{(n)} + \lambda^{T-t-1} G_t$$

Online v/s off-line G_t -lambda

It is all OFF-line $\rightarrow$ As G_t is in target $\rightarrow$ you have to wait

Converted to offline problem

- It is computed at the end of trajectory
- Online, offline don't make sense here

Min achieved for $\lambda = 0.9$

- Similar curve

What is the notion of online - they did TD version

- Will show online version of TD-lambda

So far we can appreciate, online vs offline

Backward view will give online update

w7L40 Backward View Eligibility Traces

Backward View.

$$E_t(s) = \begin{cases} (\gamma \lambda) E_{t-1}(s) & \text{if } s_t \neq s \\ (\gamma \lambda) E_{t-1}(s) + 1 & \text{if } s_t = s \end{cases}$$

$$E_t(s) = \begin{cases} (\gamma \lambda) E_{t-1}(s) & \text{if } s_t \neq s \\ (\gamma \lambda) E_{t-1}(s) + 1 & \text{if } s_t = s \end{cases}$$

Accumulating Traces.

Replacing Traces.

$1 \text{ if } s_t = s$



Go upto 1

$$\delta_t = R_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

$$\Delta V_t(s) = \alpha \cdot \delta_t \cdot E_t(s)$$

delta -t is time index NOT state index

- Will use to change values of all states

$$G_t = (1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_t^{(n)} + \lambda G_t$$

All will telescope into each other, we will end up computing G_t -lambda

- That's the cool version

On-line

off-line

$$\Delta V_t(s) = \alpha [G_{t+1}^{(n)} - V_t(s)]$$

$$V_{t+1}(s) = V_t(s) + \Delta V_t(s)$$

$$V(s) = V(s) + \sum_t \Delta V_t(s)$$

Have both online and offline version -in book, diagram 0.8 and 0.6

- Generated for different steps of lambda

9min:

What will the TD-lambda update look like

- Update eligibility trace for all states

In offline - compute change in value fn, in accumulator,

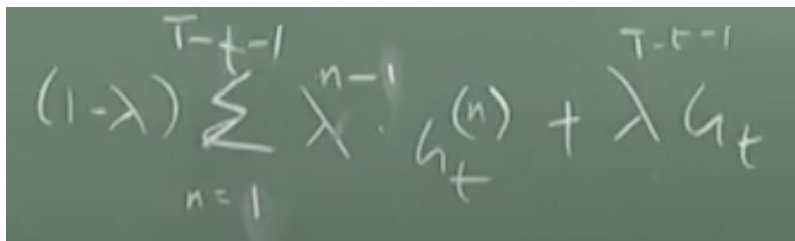
At the end of episode make update

If lambda decays very quickly to close to zero

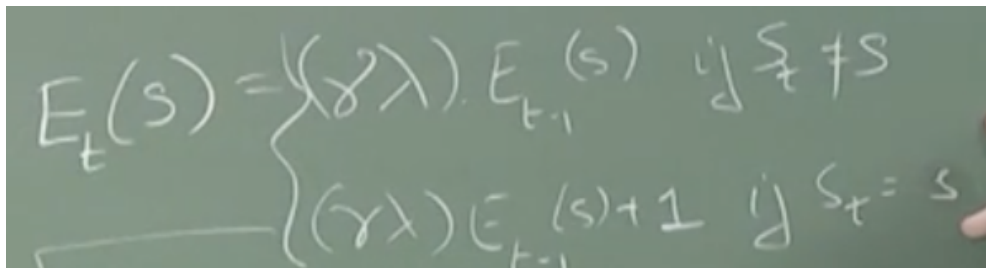
For very large space - don't want to update every step

- At some point make it zero

If make lambda =0, it is TD(0)


$$(1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_{t+n} + \lambda G_{t+T-t}$$

If lambda =1 -> we are left with G_t - full return, so becomes MC


$$E_t(s) = \begin{cases} (\gamma\lambda) \cdot E_{t-1}(s) & \text{if } s \neq S_t \\ (\gamma\lambda) \cdot E_{t-1}(s) + 1 & \text{if } s = S_t \end{cases}$$

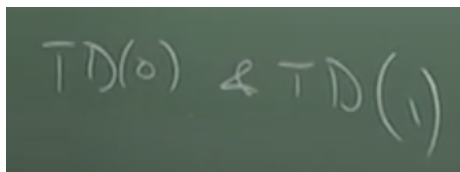
Have incremental way of updating MC

Gives way to do MC update for infinite trajectories (non-episodic tasks)

What is the problem with infinite trajectories

- Too large, so what
- Make sure gamma is large (or 1)
- TD(lambda) - will have to make sure non-one gamma

Special cases:


$$TD(0) \text{ \& } TD(1)$$

Have to do Forward and backward TD

- Equivalence of forward and backward
- 1st edition of book section 7.4

$$G_t^{\gamma} = (1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_{t+n}^{\gamma} + \lambda G_t^{\gamma}$$

Offline return - should not be looking at online update

On-line

off-line

$$\Delta V_t(s) \propto [G_{t+n}^{\gamma} - V_t(s)]$$

$$V_{t+1}(s) = V_t(s) + \Delta V_t(s)$$

$$V(s) = V(s) + \sum_t \Delta V_t(s)$$

Zeroing out eligibility traces after a while

- That is backward view

What is the forward view?

- It is not trivial
- Simple - make it zero after 3 steps

Backward View

$$E_t(s) = (\gamma \lambda) E_{t-1}(s) + \delta [G_t(s) - V_t(s)]$$

What is the forward eligibility view?

How we do arithmetic to do forward and backward view

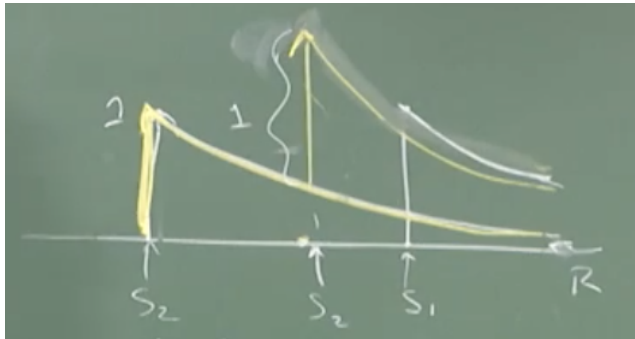
22min:

First visit and Every visit

Not much work is done on -

- For online forward view is not clean
-

Online TD (λ) = True or exact TD(λ)



Accumulating traces:

- Implements - every visit MC

Replacing traces:

- Implements First Visit MC

$$E_t(s) = \begin{cases} (\gamma \lambda) E_{t-1}(s) & \text{if } S_t \neq s \\ (\gamma \lambda) E_{t-1}(s) + 1 & \text{if } S_t = s \end{cases}$$

Accumulating Traces.

Replacing Traces.

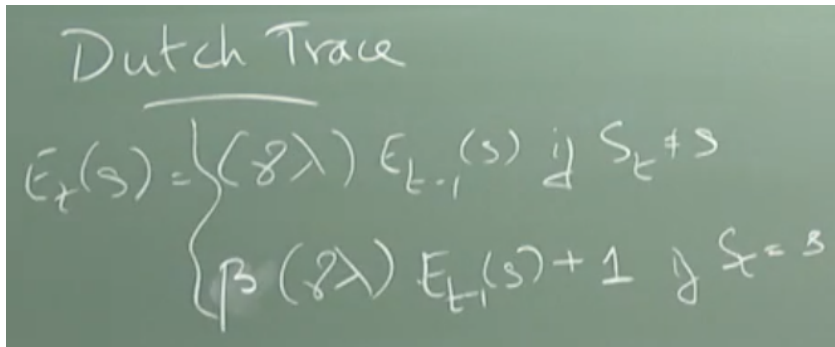
Terms - telescope in each other - some of the terms will start cancelling out

- Whatever started from beginning will continue till end

When replace 1, summation will not start

Refer 7.4 1st edition

Dutch trace:



The image shows a chalkboard with the title "Dutch Trace" underlined. Below it, the equation for $E_t(s)$ is written as a piecewise function:

$$E_t(s) = \begin{cases} (\gamma\lambda) E_{t-1}(s) & \text{if } S_t \neq s \\ \beta (\gamma\lambda) E_{t-1}(s) + 1 & \text{if } S_t = s \end{cases}$$

Beta = 1, get accumulating trace

Beta = 0, get replacing trace

Beta - other -> it is Dutch trace

Give additional parameter to tune

- Better performance with intermediate values of beta
- Largely the advantage is Empirical

Gamma - is the parameter of the **problem**

- Changing gamma changes the right answer

Lambda is parameter of solution

- Changing lambda does not change the right answer
- Optimal policy is same
- Optimal value fn is same
- Gt-lambda, Gt-MC, Gt-TD

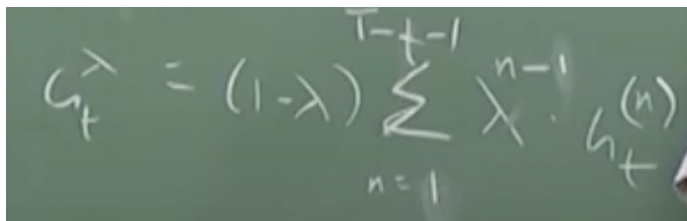
Finite sample space

- If take true space

Backward

- Gamma lambda come together

When sum up get 1 return



The image shows a chalkboard with the equation for G_t^λ written as:

$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_{t+n}$$

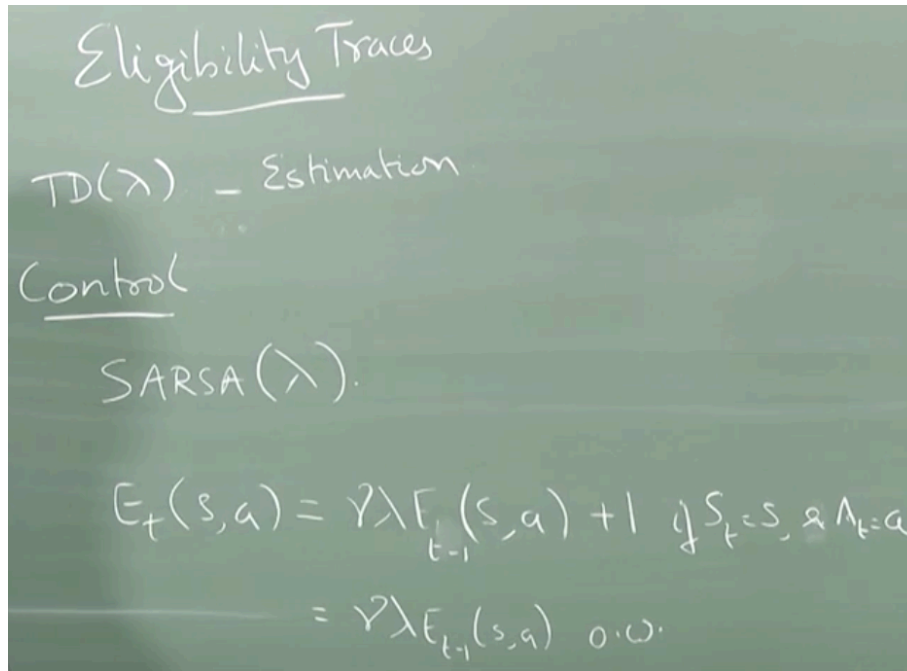
Alpha also affects the solution

- Any para will affect the solution

w7L41 Eligibility Trace Control

Eligibility trace in SARSA:

- Backward view - easy to see where we are going = TD(lambda)



Eligibility Traces

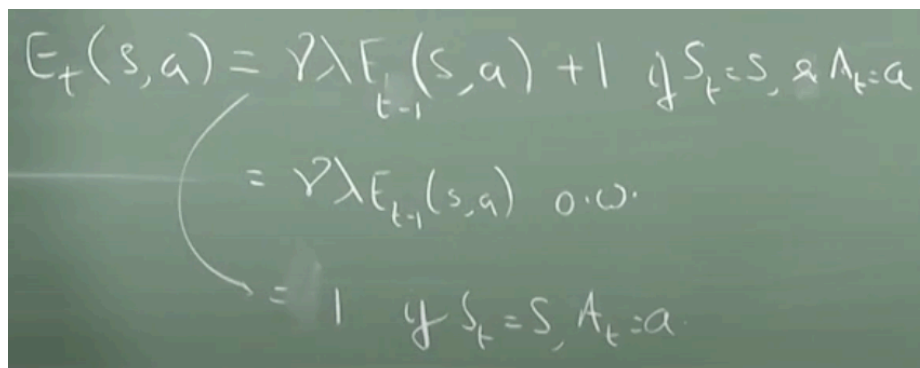
TD(λ) - Estimation

Control

SARSA(λ).

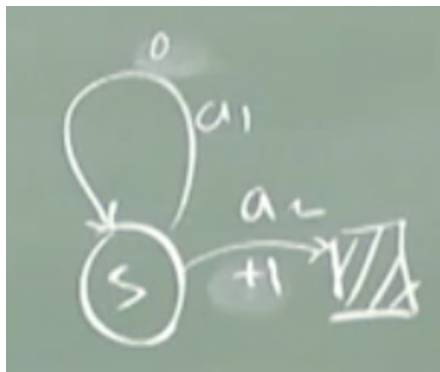
$$E_t(s, a) = \gamma \lambda E_{t-1}(s, a) + 1 \quad \text{if } S_t = s, A_t = a$$
$$= \gamma \lambda E_{t-1}(s, a) \quad \text{o.w.}$$

For accumulating trace

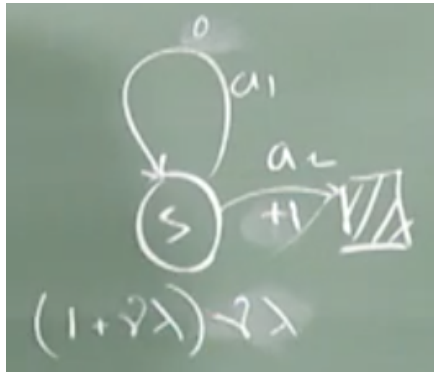

$$E_t(s, a) = \gamma \lambda E_{t-1}(s, a) + 1 \quad \text{if } S_t = s, A_t = a$$
$$= \gamma \lambda E_{t-1}(s, a) \quad \text{o.w.}$$
$$= 1 \quad \text{if } S_t = s, A_t = a.$$

For replacing trace

Careful about Eligibility traces on state and action pair



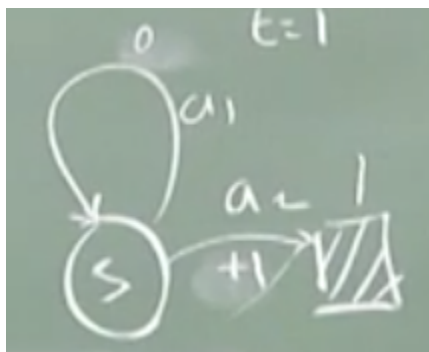
a1 many times, then a2



Every time go

$$\sum_{t=1}^K \gamma^t$$

Trace of a1. trace will be this, depending on gamma, lambda, value could be larger than 1



trace of a2 will be one

Which action will give highest -> when take a2, a1 will get rewarded

Replacing trace will FIX your problem

Aggressive:

Come to a state, take action

For all other action i could have taken - make trace =0

- That will not go back to other action taken

From a1 to a2, a1 will be zero

Visit state s, take a1, then take a2, don't want this reward to update a1

$$\begin{aligned}
 E_t(s, a) &= \begin{cases} \gamma \lambda E_{t-1}(s, a) + 1 & \text{if } S_t = s, A_t = a \\ \gamma \lambda E_{t-1}(s, a) & \text{o.w.} \end{cases} \\
 &= \begin{cases} 1 & \text{if } S_t = s, A_t = a \\ 0 & \text{if } S_t = s, A_t \neq a \end{cases} \\
 &= \gamma \lambda E_{t-1}(s, a) \cdot \mathbb{1}_{S_t = s}
 \end{aligned}$$

For that action set trace to 1, for others zero

Forward view for SARSA lambda:

$(1 - \lambda) \cdot \text{first step return} + \text{etc.}$

$$\begin{aligned}
 \Delta Q_t(s, a) &= \alpha \cdot \delta_t \cdot E_t(s, a) \\
 \delta_t &= R_{t+1} + \gamma Q_t(S_{t+1}, A_{t+1}) - Q_t(S_t, A_t)
 \end{aligned}$$

You can generalise from TD(lambda)

Instead of $v(s)$ - written $Q(s)$

Control Algo - Q(lambda):

Watkins Q(lambda) - is justifiable - doesn't work well in practice

- What will be the generalisation for Eligibility traces
- Take action and you are assuming, you will behave optimally subsequently

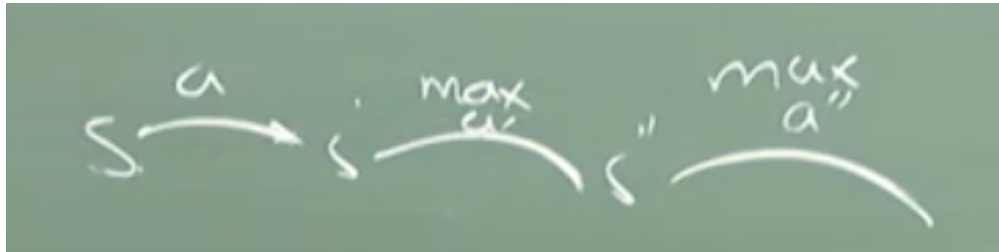
In SARSA - whatever action was taken

$$Q_t = R_{t+1} + \gamma \max_{a'} Q_t(s_{t+1}, a') - Q_t(s_t, A_t)$$

In Q learning optimal action

While computing action,

- Behave greedily - i can use entire trajectory for learning
- With epsilon greedy -> return i am generating may not be optimal policy



When use greedy policy, use entire return to update policy

Why we need to be greedy - have greedy trajectory

- Assumption of Q learning

We need to explore

- Otherwise we will stuck in suboptimal policy

How do I balance?

With lambda taking weighted return -> imagine amount of memory you will need

Eligibility trace for short period of time

You do averaging till n steps

- Only upto 4 states, then do exploration
- State after that will be 3 update, state 2, state 1, s' will get regular Q-learning update
- That is tricky part for Forward
- Will not write it

What changes is this:

$$E_t(s, a) = \gamma \lambda E_{t-1}(s, a) + 1 \quad \text{if } S_t = s \text{ \& } A_t = a$$
$$A_t = \arg \max_{a'} Q_t(s_t, a')$$
$$= \gamma \lambda E_{t-1}(s, a) \quad \text{if } A_t \neq a \text{ \& } A_t = \arg \max_{a'} Q_t(s_t, a')$$

Increment by one

$$E_t(s, a) = \gamma \lambda E_{t-1}(s, a) + 1 \quad \text{if } S_t = s \text{ \& } A_t = a$$
$$A_t = \arg \max_{a'} Q_t(s_t, a')$$
$$= \gamma \lambda E_{t-1}(s, a) \quad \text{if } A_t \neq a \text{ \& } A_t = \arg \max_{a'} Q_t(s_t, a')$$
$$= 0 \quad \text{if } A_t \neq \arg \max_{a'} Q_t(s_t, a')$$

Last step is exploring option

For greedy 1st line

Watkins - all traces are very short

Epsilon 0.1 -> 10% time exploratory

- Loose adv of doing eligibility traces



With regular Q-learning or SARSA which states will change

- Online - only the last action (right yellow marked cell)



With SARSA(lambda) - all states

- It converges faster

Plain sarsa, plain Q - learning only one state

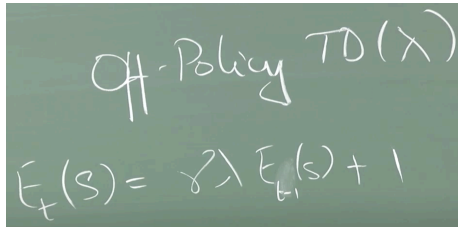
What does Q learning do? It is called off policy

- We don't know what optimal policy (π - optimal)
- μ - we are following it

Off policy version of TD(λ) - trajectories coming from μ

You will never be running SARSA - Offline

Online version - equivalence between forward and backward view

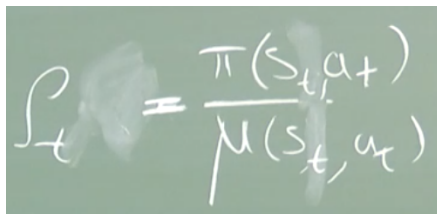


Handwritten text: Off-Policy TD(λ)

$$E_t(s) = \gamma \lambda E_{t-1}(s) + 1$$

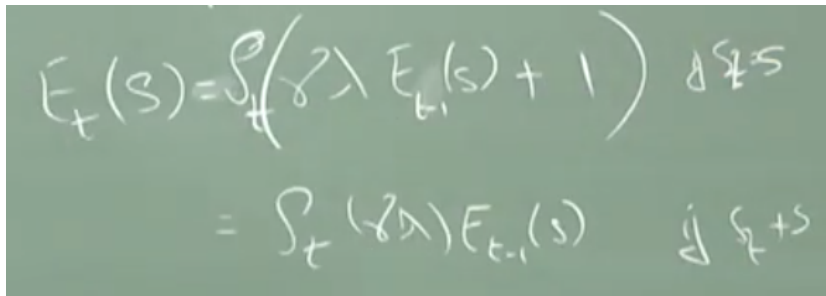
How will we insert - importance weights into it?

Define ρ_t


$$\rho_t = \frac{\pi(s_t, a_t)}{\mu(s_t, a_t)}$$

ref- note from importance sampling

What do i multiply


$$\begin{aligned} E_t(s) &= \rho_t \left(\gamma \lambda E_{t-1}(s) + 1 \right) \quad \text{if } s_t = s \\ &= \rho_t (\gamma \lambda) E_{t-1}(s) \quad \text{if } s_t \neq s \end{aligned}$$

w7L42 Thompson Sampling Recap

What is the model parameter - solve

MLE, MAP - will find one estimate for para- θ

- What is the process that generated that data

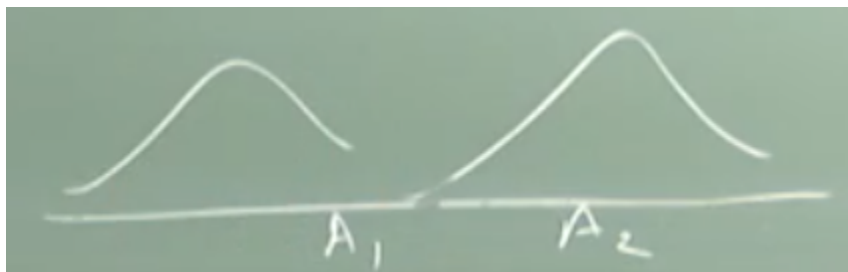
Translate to bandit setting

- Arm pulls and rewards
- Figure out something about the para
- Then solve para

Value fn

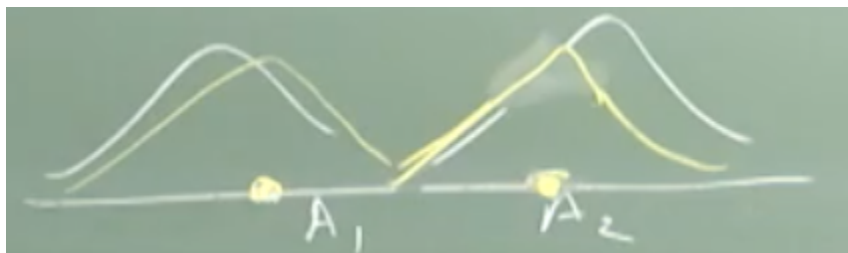
- Take data
- Make guess about the problem (MLE) i am solving
- MLE = average reward for all arms

MAP:



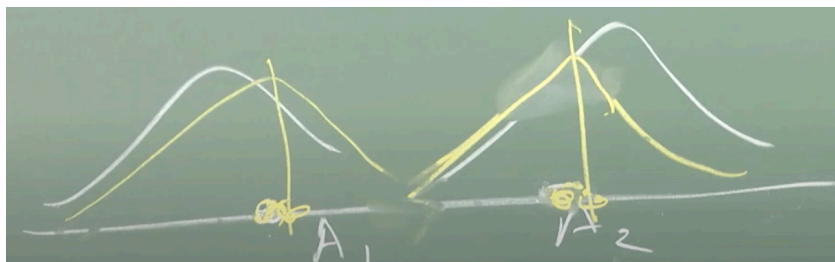
Don't know what μ_1 and μ_2 is

Real μ_1 in this range,



Pull Arm-a get reward here, pull arm-b get reward here

MAP - assume something about prior means,



Which para has highest probability

- find max prob in posterior
 - Prob that the parameter is right para

Full Bayesian version

- Will not pick one value for my mean
- Assume all values are likely, likelihood is this distribution
 - This is the distribution over mean



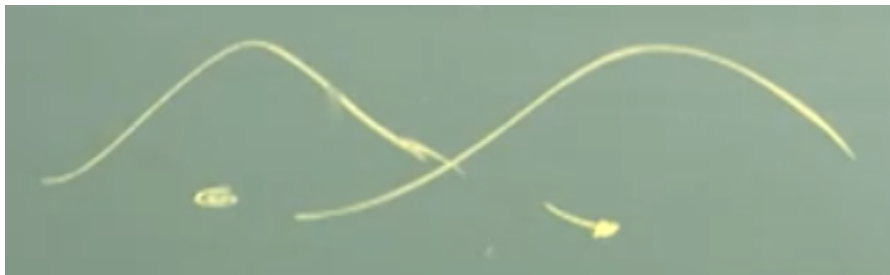
- When i pull arm, will have different distribution
 - Thin smaller bottom curve
- If mean is here, get some reward
- Will have different variance

Pull arm which every gives best expectation

- μ comes from a certain distribution
- Expected Reward for that μ value
- What is the probability that μ is correct

Approximate this estimation:

- Take posterior
- Draw samples
- Assume samples are mean
- Go and solve problem according to that
- Now will get some data point
- Change posterior
- Keep doing it



In effect you are trying to do Bayesian version
We have ML and bayesian version of bandit problem

13min: Looks like Repeat:

- What is the model para
- Trying to solve
- Find one estimate, solve problem corresponding to that θ
- What is the process that generated the data

Translating that to bandit setting:

- Based on bandit -
- Make guess about the problem - MLE guess
- I have no other information - take average of all rewards got for arm-A

For MAP assumption:

- Assume something about priors of mean
- Find posterior

Prob of getting rewards

Prob that the para is right version

Full version: